

سوال یک :

رعایت اصل SRP باعث می شود :

- هر کلاس یک وظیفه مشخص و محدود داشته باشد، بنابراین:
- کد خواناتر و قابل فهم تر می شود.
- احتمال ایجاد کلاس های بزرگ و پراکنده با خطوط زیاد کاهش می یابد.
- ساختار کد منظم و متمرکز می شود؛ در نتیجه:
- تغییرات را می توان در یک بخش اعمال کرد بدون اینکه روی بخش های دیگر اثر بگذارد.
- نگهداری، تست و توسعه کد ساده تر و ایمن تر می شود.
- رعایت SRP به کاهش وابستگی ها، افزایش قابلیت استفاده مجدد و تسهیل تست ها کمک می کند.

مثال :

```
public class Invoice
{
    public void AddInvoice()
    {
        // logic
    }

    public void DeleteInvoice()
    {
        // logic
    }

    public void GenerateReport()
    {
        // logic
    }
}
```

در مثال بالا فرآیند ایجاد و حذف فاکتور به همراه فرآیند گزارش گیری در کلاس فاکتور نوشته شده که اشتباه است و باید به

صورت زیر از هم تفکیک شوند.

```
public class Invoice
{
    public void AddInvoice()
    {
        // logic
    }
}
```

```

        public void DeleteInvoice()
        {
            // logic
        }
    }

    public class Report
    {
        public void GenerateReport()
        {
            // logic
        }
    }

```

سوال دو :

رعایت اصل OCP باعث می شود :

اصل OCP یعنی امکان افزودن قابلیت جدید بدون تغییر در کد موجود. یعنی باز برای توسعه و بسته در برابر تغییر.

- افزودن قابلیت‌های جدید بدون تغییر کد موجود ممکن می‌شود.
- تغییر کمتر در کد پایدار، احتمال ورود باگ جدید را کم می‌کند.
- جایگزینی یا افزودن رفتارها با کمترین تغییر ممکن می‌شود.
- افزودن ویژگی‌ها سریع‌تر و با ریسک کمتر انجام می‌شود.
- مقیاس پذیری و قابلیت نگهداری کد بیشتر می‌شود.

مثال :

```

public class Circle { }
public class Square { }
public static class Drawer {
    public static void DrawShapes(IEnumerable<object> shapes) {
        foreach (object shape in shapes) {
            if (shape is Circle) { DrawCircle(shape as Circle); }
            else if (shape is Square) { DrawSquare(shape as Square); }
        }
    }
    private static void DrawCircle(Circle circle) { /*Draw circle*/ }
    private static void DrawSquare(Square square) { /*Draw Square*/ }
}

```

در صورتی که بخواهیم شکل جدیدی اضافه کنیم باید کد اصلی را تغییر دهیم که خلاف این اصل می باشد.

```
public interface IShape { void Draw(); }

public class Circle : IShape { public void Draw() { /*Draw circle*/ }}

public class Square : IShape { public void Draw() { /*Draw Square*/ } }

public static class Drawer {
    public static void DrawShapes(IEnumerable<IShape> shapes) {
        foreach (IShape shape in shapes) {
            shape.Draw();
        }
    }
}
```

سوال سه :

اصل LSP :

کلاس های فرزند باید بتوانند بدون تغییر رفتار جایگزین کلاس های والد شوند

- توسعه آسان : توسعه دهندگان می توانند کلاس های مشتق شده جدید را با اطمینان ایجاد کنند و تغییرات روی سیستم را با کمترین تلاش اعمال کنند.

- توسعه دهندگان می توانند بدون نیاز به تغییرات گسترده در کد موجود به راحتی تغییرات و بهبودها روی آن ها اعمال کنند.

```
public class Rectangle
{
    public virtual double Width { get; set; }
    public virtual double Height { get; set; }
    public double GetArea()
    {
        return Width * Height;
    }
}

public class Square : Rectangle
{
    public double Width {
        set {
            base.Width = value;
            base.Height = value;
        }
    }
}
```

```

    public double Height {
        set
        {
            base.Width = value;
            base.Height = value;
        }
    }
}

```

کد اصلاحی:

```

public abstract class Shape
{
    public abstract double GetArea();
}

public class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }
    public override double GetArea()
    {
        return Width * Height;
    }
}

public class Square : Shape
{
    public double Width { get; set; }
    public override double GetArea()
    {
        return Width * Width;
    }
}

```

سوال چهار :

اصل ISP :

یک واسط نباید شامل متدهایی باشد که یک کلاس مجبور به پیاده سازی آنها بدون نیاز واقعی شود.

- کاهش وابستگی کلاس ها به متد های غیرضروری
- انعطاف پذیری بیشتر در تغییرات

```

public interface IInvoice
{
    public void Create();
    public void Update();
    public void Delete();
    public void OnlinePayment();
    public void CashPayment();
    public void ChequePayment();
}

public class Invoice : IInvoice
{

```

```

    public void Create()
    {
        // logic
    }

    public void Delete()
    {
        // logic
    }

    public void Update()
    {
        // logic
    }

    public void CashPayment()
    {
        // logic
    }

    public void ChequePayment()
    {
        // logic
    }

    public void OnlinePayment()
    {
        // No Implementation
    }
}

public class OnlineInvoice : IInvoice
{
    public void Create()
    {
        // logic
    }

    public void Delete()
    {
        // logic
    }

    public void Update()
    {
        // logic
    }

    public void CashPayment()
    {
        // No Implementation
    }

    public void ChequePayment()
    {
        // No Implementation
    }

    public void OnlinePayment()
    {
        // logic
    }
}

```

```

public interface IInvoice
{
    public void Create();
    public void Update();
    public void Delete();
}

public interface IPayment
{
    public void CashPayment();
    public void ChequePayment();
}

public interface IOnlinePayment
{
    public void OnlinePayment();
}

public class Invoice : IInvoice, IPayment
{
    public void Create()
    {
        // logic
    }

    public void Delete()
    {
        // logic
    }

    public void Update()
    {
        // logic
    }

    public void CashPayment()
    {
        // logic
    }

    public void ChequePayment()
    {
        // logic
    }
}

public class OnlineInvoice : IInvoice, IOnlinePayment
{
    public void Create()
    {
        // logic
    }

    public void Delete()
    {
        // logic
    }

    public void Update()

```

```

{
    // logic
}

public void OnlinePayment()
{
    // logic
}
}

```

سوال پنج :

اصل DIP :

ماژول های سطح بالا نباید به ماژول های سطح پایین وابسته باشند. هر دو باید به واسطه وابسته باشند.

- افزایش قابلیت توسعه.
- کاهش وابستگی کلاس ها به هم.

```

public class EmailSender
{
    public void SendEmail(string email)
    {
        // logic
    }
}

public class PaymentService
{
    private EmailSender _emailSender = new EmailSender();
    public void Pay()
    {
        //logic
        _emailSender.SendEmail("Payed");
    }
}

```

کد اصلاحی :

```

public interface INotification
{
    public void Send(string message);
}

public class EmailSender: INotification
{
    public void Send(string email)
    {
        // logic
    }
}

public class SmsSender : INotification
{

```

```
        public void Send(string sms)
        {
            // logic
        }
    }

    public class PaymentService
    {
        private readonly INotification _notification;

        public PaymentService(INotification notification)
        {
            _notification = notification;
        }

        public void Pay()
        {
            //logic
            _notification.Send("Payed");
        }
    }
}
```

سوال ۶:

گزینه B چنانچه گزینه C هم می تواند در مواردی این اصل را نقض کند